14 · Foundations: app setup, database, data loading, shared UI

The pieces every feature relies on, on both sides.

New to the syntax? Read **00 · How to read this code** first; this guide explains what is specific to this feature.

What this feature does

This guide covers the shared code the feature guides keep pointing to: how the backend app starts and mounts its routes, how a database session reaches a route, the model base classes, the "who is calling" checks, how the frontend sends requests and loads data, the route table, and the shared UI pieces.

Files involved

Only the files this feature uses.

```
backend/app/
├── main.py                 creates the app, CORS, mounts /api/v1 and /api/v1/media
├── api/router.py           joins every route file
├── db/session.py           engine + get_session()
├── models/base.py          Base, UUIDPk, CreatedAt, utcnow()
├── api/views.py            alert_out(), person(), visit_for_*(), checks_of()
└── core/dependencies.py    get_current_user, require_user, require_phc_worker,
                            require_any_verified...
frontend/src/
├── main.tsx                the route table (which page, which URL, which role)
├── services/api.ts         the shared axios client
├── lib/useResource.ts      loading data for a page
├── lib/params.ts           filters in the URL
└── components/app/ui.tsx   PageHeader, Card, Btn, Stat, Tabs, Loading, ErrorNote, Empty, badges
```

Code walkthrough

Every excerpt below is copied straight from the file, with its real line numbers, when this guide was generated.

1. `backend/app/main.py` · main.py (app setup)

```
26  app = FastAPI(title="HealthNexus API", version="0.1.0", lifespan=lifespan)
27
28  app.add_middleware(
29      CORSMiddleware,
30      allow_origins=get_settings().cors_origins,
31      allow_credentials=True,
32      allow_methods=["*"],
33      allow_headers=["*"],
34  )
35
36  app.include_router(api_router, prefix="/api/v1")
37
38  # Uploaded profile images. File names are random, so a URL cannot be guessed from an id.
39  media_root().mkdir(parents=True, exist_ok=True)
40  app.mount(MEDIA_URL, StaticFiles(directory=media_root()), name="media")
41
```

What the code means

- `FastAPI(...)` creates the application; `lifespan` runs checks at start-up (the server refuses to start without a strong secret).
- `CORSMiddleware` allows the frontend's address to call the API from the browser.
- `app.include_router(api_router, prefix="/api/v1")` puts every endpoint under `/api/v1`.
- `app.mount(MEDIA_URL, StaticFiles(...))` serves uploaded images straight from a folder.

2. `backend/app/api/router.py` · router.py

```python
from fastapi import APIRouter

from app.api.routes import alerts, auth, onboarding, phc, profile, schedule, settings, vitals

api_router = APIRouter()
api_router.include_router(auth.router)
api_router.include_router(onboarding.router)
api_router.include_router(vitals.router)
api_router.include_router(alerts.router)
api_router.include_router(alerts.phc_router)
api_router.include_router(schedule.router)
api_router.include_router(phc.router)
api_router.include_router(settings.router)
api_router.include_router(profile.router)
```

What the code means

- Each route file defines its own `APIRouter` with a prefix (for example `/health`); this file adds them all to one router.

3. `backend/app/db/session.py` · db/session.py

```python
engine = create_async_engine(get_settings().DATABASE_URL, pool_pre_ping=True)
SessionLocal = async_sessionmaker(engine, expire_on_commit=False)


async def get_session() -> AsyncIterator[AsyncSession]:
    async with SessionLocal() as session:
        yield session
```

What the code means

- `create_async_engine(URL, pool_pre_ping=True)` opens a pool of connections to PostgreSQL and checks a connection is alive before using it.
- `get_session` is a dependency: `async with` opens a session, `yield` hands it to the route, and it is closed when the request ends, even after an error.
- Tables are created by Alembic migrations, never by the app.

4. `backend/app/models/base.py` · models/base.py

```python
class Base(DeclarativeBase):
    metadata = MetaData(naming_convention=NAMING_CONVENTION)
    type_annotation_map = {datetime: DateTime(timezone=True)}


class UUIDPk:
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)


class CreatedAt:
    created_at: Mapped[datetime] = mapped_column(default=utcnow)

```

What the code means

- `Base` is the parent of every table class; `type_annotation_map` makes every `datetime` column timezone-aware.
- `UUIDPk` and `CreatedAt` are **mixins**: small classes that add a column to any model that inherits them, for example `class HealthRecord(UUIDPk, CreatedAt, Base)`.

5. `backend/app/core/dependencies.py` · require_any_verified

```
async def require_any_verified(user: User = Depends(get_current_user)) -> User:
    """Any approved role, with a confirmed email. A pending_phc account is not yet approved."""
    if user.role not in APP_ROLES:
        raise _forbidden("Your account is not active yet.")
    if not user.is_verified:
        raise _forbidden("Verify your email address to continue.")
    return user
```

What the code means

- A dependency built on `get_current_user` (which reads the login token and loads the user fresh from the database).
- It refuses accounts that are not approved yet and emails that are not verified. Most shared routes (alerts, settings, profile) use it.

6. `backend/app/api/views.py` · person

```
def person(citizen: User) -> VisitPerson:
    return VisitPerson(id=citizen.id, ref=citizen_ref(citizen.id), name=citizen.name, phone=citizen.phone, a
        rea=citizen.area)
```

What the code means

- Builds the short person summary used on many worker pages, including the stable `HN-XXXXXXXX` reference.

7. `frontend/src/services/api.ts` · the axios client

```
export const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL ?? "/api/v1",
  withCredentials: true, // sends the hn_refresh cookie
});

api.interceptors.request.use((config) => {
  if (accessToken) config.headers.set("Authorization", `Bearer ${accessToken}`);
  return config;
});
```

What the code means

- `baseURL` is `/api/v1` unless `VITE_API_URL` is set; `withCredentials: true` sends the refresh cookie.
- `interceptors.request.use(...)` runs before every request and adds `Authorization: Bearer <token>`. (Refreshing an expired token is explained in the Auth guide.)

8. `frontend/src/lib/useResource.ts` · useResource

```
export function useResource<T>(fetcher: (signal: AbortSignal) => Promise<T>, key: string): Resource<T> {
  const [data, setData] = useState<T | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [status, setStatus] = useState<number | null>(null);
  const [loading, setLoading] = useState(true);
  const [tick, setTick] = useState(0);
  const latest = useRef(0);
  const fetcherRef = useRef(fetcher);
  fetcherRef.current = fetcher;

  useEffect(() => {
    const id = ++latest.current;
    const controller = new AbortController();
    setLoading(true);
    fetcherRef
      .current(controller.signal)
      .then((result) => {
        if (id !== latest.current) return;
        setData(result);
        setError(null);
        setStatus(null);
      })
      .catch((e: unknown) => {
        if (id !== latest.current || controller.signal.aborted) return;
```

```
        setError(errorMessage(e));
        setStatus(e instanceof ApiError ? e.status : 0);
      })
      .finally(() => {
        if (id === latest.current) setLoading(false);
      });
    return () => controller.abort();
  }, [key, tick]);

  const reload = useCallback(() => setTick((t) => t + 1), []);
  return { data, error, status, loading, reload };
}
```

What the code means

- Returns `{ data, error, status, loading, reload }` for any page. `<T>` keeps the type of the data.
- `latest` counts requests; an answer is used only if it belongs to the newest request, so a slow old answer cannot overwrite a newer one.
- `AbortController` cancels the request when the component goes away or the key changes (the effect's clean-up).
- `fetcherRef` always holds the newest fetcher without making it a dependency; only `key` and `tick` trigger a reload.
- `reload()` bumps `tick`, which re-runs the effect.

9. `frontend/src/lib/params.ts` · useQueryState

```
export function useQueryState() {
  const [params, setParams] = useSearchParams();
  const set = useCallback(
    (updates: Record<string, string | number | null | undefined>) => {
      setParams(
        (prev) => {
          const next = new URLSearchParams(prev);
          for (const [key, value] of Object.entries(updates)) {
            if (value === null || value === undefined || value === "") next.delete(key);
            else next.set(key, String(value));
          }
          return next;
        },
        { replace: true },
      );
    },
    [setParams],
  );
  return useMemo(() => ({ params, set }), [params, set]);
}
```

What the code means

- Wraps React Router's `useSearchParams`. `set({ page: 2, q: null })` updates several URL parameters at once; empty values remove the key.
- `{ replace: true }` replaces the browser history entry, so changing filters does not fill the Back button.

10. `frontend/src/main.tsx` · main.tsx (citizen and worker routes)

```
            <Route element={<ProtectedRoute allowedRoles={["user"]} />}>
              <Route element={<AppShell />}>
                <Route path={paths.citizenHome} element={<CitizenDashboard />} />
                <Route path={paths.submitVitals} element={<SubmitVitals />} />
                <Route path={paths.history} element={<History />} />
                <Route path={paths.citizenAlerts} element={<CitizenAlerts />} />
              </Route>
            </Route>
            <Route element={<ProtectedRoute allowedRoles={["phc_worker"]} />}>
              <Route element={<AppShell />}>
                <Route path={paths.phcHome} element={<PhcDashboard />} />
                <Route path={paths.phcAlerts} element={<PhcAlerts />} />
                <Route path={paths.phcCitizens} element={<Citizens />} />
                <Route path="/phc/citizens/:userId" element={<CitizenRecord />} />
                <Route path={paths.phcSchedule} element={<Schedule />} />
```

```
109                 <Route path={paths.phcReports} element={<Reports />} />
110                 <Route path={paths.phcMap} element={<RiskMap />} />
111                 <Route path={paths.notifications} element={<Notifications />} />
```

What the code means

- `<Route element={<ProtectedRoute allowedRoles={["user"]} />}>` wraps a group of pages that only citizens may open.
- `<Route element={<AppShell />}>` gives those pages the shared menu; each `<Route path=... element=... />` maps a URL to a page.
- `/phc/citizens/:userId` — `:userId` is a URL parameter the page reads with `useParams()`.

11. `frontend/src/components/app/ui.tsx` · Stat

```
126 export function Stat({ label, value, hint, tone }: { label: string; value: ReactNode; hint?: ReactNode; tone
        ?: "danger" | "warn" }) {
127   return (
128     <div className="rounded-2xl border border-line bg-surface p-4">
129       <p className="text-xs font-medium uppercase tracking-wide text-ink-soft">{label}</p>
130       <p className={cx("mt-1 text-2xl font-semibold tabular-nums", tone === "danger" ? "text-danger" : tone
            === "warn" ? "text-warn" : "text-ink")}>
131         {value}
132       </p>
133       {hint && <p className="mt-1 text-xs text-ink-soft">{hint}</p>}
134     </div>
135   );
136 }
```

What the code means

- One of the shared UI pieces: a number card with a label and a hint. `tone` colours the number for danger or warning.
- `cx(...)` joins class names and skips empty ones, so classes can be added conditionally.

Syntax used in this feature

Code	What it means
`app.include_router(r, prefix=...)`	Mount a group of endpoints under a path.
`async with X() as s: yield s`	Open, hand over, and always close.
`class A(Mixin1, Mixin2, Base)`	Inherit columns from several small classes.
`axios.interceptors.request.use(f)`	Run f before every request.
`useSearchParams()`	Read and write the URL's ?query part.
`<Route path="/x/:id">`	A page URL with a parameter.